

Week 9

1. To Implement how to List Active Jobs, Display Status Information, Retrieve Job IDs, Format Output, Update State Information, Test Job Listings

```
root@DUDDUKURIs x + v
GNU nano 8.7.1 src/list_jobs.c *
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

#define MAX_JOBS 5

typedef struct {
    int job_id;
    pid_t pid;
    char state[20];
} Job;

Job jobs[MAX_JOBS];
int job_count = 0;

void add_job(pid_t pid) {
    jobs[job_count].job_id = job_count + 1;
    jobs[job_count].pid = pid;
    snprintf(jobs[job_count].state, 20, "Running");
    job_count++;
}

void update_jobs() {
    for (int i = 0; i < job_count; i++) {
        if (waitpid(jobs[i].pid, NULL, WNOHANG) > 0)
            snprintf(jobs[i].state, 20, "Completed");
    }
}

void list_jobs() {
    update_jobs();

    printf("\nActive Jobs:\n");
    printf("Job ID\tPID\tStatus\n");

    for (int i = 0; i < job_count; i++) {
        if (jobs[i].state[0] != 'C') {
            printf("[%d]\t%d\t%s\n",
                jobs[i].job_id,
```

```

int main() {
    pid_t pid1, pid2;

    pid1 = fork();

    if (pid1 == 0) {
        sleep(5);
        return 0;
    }

    add_job(pid1);

    pid2 = fork();

    if (pid2 == 0) {
        sleep(8);
        return 0;
    }

    add_job(pid2);

    printf("Two background jobs started.\n");

    list_jobs();

    sleep(6);

    printf("\nAfter updating job states:\n");
    list_jobs();

    waitpid(pid2, NULL, 0);

    return 0;
}

```

Output:

```

root@DUDDUKURIs  ×  +  ∨
root@DUDDUKURIs:~$ cd ~/OS_Skill
mkdir -p Skill-Week-9/src Skill-Week-9/bin
cd Skill-Week-9
root@DUDDUKURIs:~/OS_Skill/Skill-Week-9$ ls
bin  src
root@DUDDUKURIs:~/OS_Skill/Skill-Week-9$ nano src/list_jobs.c
root@DUDDUKURIs:~/OS_Skill/Skill-Week-9$ gcc src/list_jobs.c -o bin/
list_jobs
root@DUDDUKURIs:~/OS_Skill/Skill-Week-9$ ./bin/list_jobs
Two background jobs started.

Active Jobs:
Job ID  PID      Status
[1]     602     Running
[2]     603     Running

After updating job states:

Active Jobs:
Job ID  PID      Status
[2]     603     Running

```

2.To design how to Identify Target Job, Transfer Terminal Control, Resume Processes, Wait for Completion, Update Job State, Test Foreground Switching

```
root@DUDDUKURIs  ×  +  v
GNU nano 8.7.1      src/foreground_switch.c *
#include <stdio.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>
#include <sys/types.h>

int main() {
    pid_t pid;
    int status;

    printf("Starting background job...\n");

    pid = fork();

    if (pid < 0) {
        perror("fork");
        return 1;
    }

    if (pid == 0) {
        setpgid(0, 0);

        printf("Child: Job started. PID = %d\n", getpid());

        raise(SIGSTOP);

        printf("Child: Job resumed in foreground.\n");

        sleep(3);

        printf("Child: Job completed.\n");

        return 0;
    }

    setpgid(pid, pid);
```

```

printf("Parent: Job PID = %d\n", pid);
printf("Parent: Job is stopped.\n");

waitpid(pid, &status, WUNTRACED);

if (WIFSTOPPED(status))
    printf("Parent: Job state = Stopped\n");

printf("\nSwitching job to foreground...\n");

kill(pid, SIGCONT);

printf("Parent: Job resumed.\n");

waitpid(pid, &status, 0);

if (WIFEXITED(status))
    printf("Parent: Job completed.\n");

return 0;
}

```

Output:

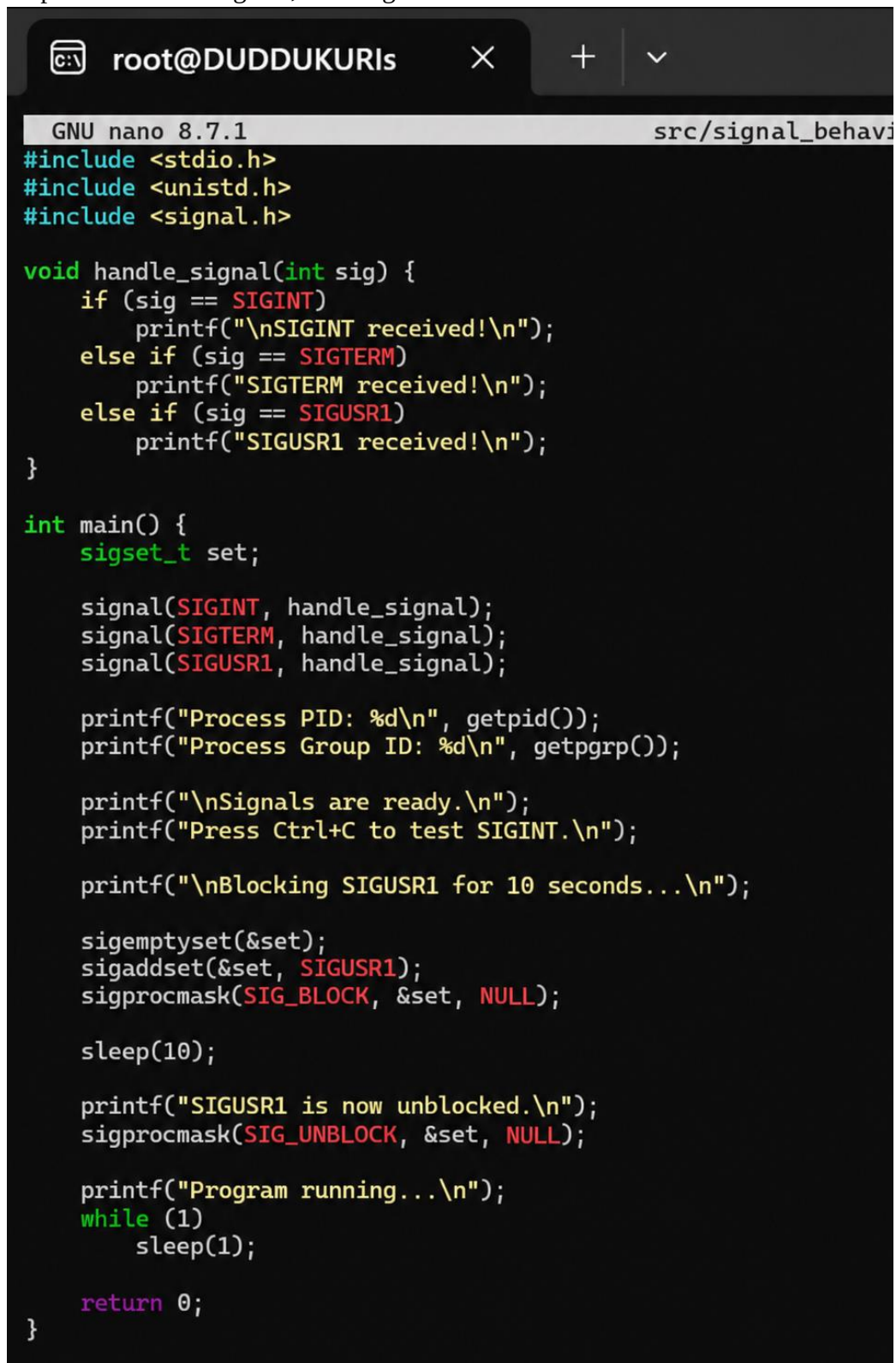
```

root@DUDDUKURIs:~/OS_Skill/Skill-Week-9$ nano src/foreground_switch.c
root@DUDDUKURIs:~/OS_Skill/Skill-Week-9$ gcc src/foreground_switch.c -o
bin/foreground_switch
root@DUDDUKURIs:~/OS_Skill/Skill-Week-9$ gcc src/foreground_switch.c -o
bin/foreground_switch
root@DUDDUKURIs:~/OS_Skill/Skill-Week-9$ ./bin/foreground_switch
Starting background job...
Parent: Job PID = 662
Parent: Job is stopped.
Child: Job started. PID = 662
Parent: Job state = Stopped

Switching job to foreground...
Parent: Job resumed.
Child: Job resumed in foreground.
Child: Job completed.
Parent: Job completed.

```

3. To Analyze Signal Delivery, Understand Signal Masks, Review Process Groups, Explore Terminal Signals, Test Signal Behavior



The screenshot shows a terminal window with a dark background. At the top, there's a tab labeled 'root@DUDDUKURIs' with a close button (X) and window control buttons (+ and v). Below the tab, the terminal title bar shows 'GNU nano 8.7.1' on the left and 'src/signal_behavi' on the right. The main area contains C code for a signal handling program. The code includes headers for stdio, unistd, and signal. It defines a 'handle_signal' function that prints messages for SIGINT, SIGTERM, and SIGUSR1. The 'main' function sets up a signal set, registers the handler for the three signals, prints process information, and then demonstrates blocking and unblocking SIGUSR1 using sigemptyset, sigaddset, sigprocmask, and sleep functions. It ends with a while loop that sleeps for 1 second and returns 0.

```
GNU nano 8.7.1 src/signal_behavi
#include <stdio.h>
#include <unistd.h>
#include <signal.h>

void handle_signal(int sig) {
    if (sig == SIGINT)
        printf("\nSIGINT received!\n");
    else if (sig == SIGTERM)
        printf("SIGTERM received!\n");
    else if (sig == SIGUSR1)
        printf("SIGUSR1 received!\n");
}

int main() {
    sigset_t set;

    signal(SIGINT, handle_signal);
    signal(SIGTERM, handle_signal);
    signal(SIGUSR1, handle_signal);

    printf("Process PID: %d\n", getpid());
    printf("Process Group ID: %d\n", getpgrp());

    printf("\nSignals are ready.\n");
    printf("Press Ctrl+C to test SIGINT.\n");

    printf("\nBlocking SIGUSR1 for 10 seconds...\n");

    sigemptyset(&set);
    sigaddset(&set, SIGUSR1);
    sigprocmask(SIG_BLOCK, &set, NULL);

    sleep(10);

    printf("SIGUSR1 is now unblocked.\n");
    sigprocmask(SIG_UNBLOCK, &set, NULL);

    printf("Program running...\n");
    while (1)
        sleep(1);

    return 0;
}
```

Output:

```
root@DUDDUKURIs  X  +  v  -  □  X
root@DUDDUKURIs:~$ cd ~/OS_Skill
root@DUDDUKURIs:~/OS_Skill$ cd Skill-Week-9
root@DUDDUKURIs:~/OS_Skill/Skill-Week-9$ gcc src/signal_
_behavior.c -o bin/signal_behavior
root@DUDDUKURIs:~/OS_Skill/Skill-Week-9$ ./bin/signal_b
ehavior
Process PID: 1090
Process Group ID: 1090

Signals are ready.
Press Ctrl+C to test SIGINT.

Blocking SIGUSR1 for 10 seconds...
^C
SIGINT received!
SIGUSR1 is now unblocked.
Program running...
SIGUSR1 received!
SIGTERM received!
```

```
root@DUDDUKURIs  X  +  v  -  □  X
root@DUDDUKURIs:~$ cd ~/OS_Skill
root@DUDDUKURIs:~/OS_Skill$ cd Skill-Week-9
root@DUDDUKURIs:~/OS_Skill/Skill-Week-9$ kill -SIGUSR1
1090
root@DUDDUKURIs:~/OS_Skill/Skill-Week-9$ kill -SIGTERM
1090
```

4. To implement how to resume Stopped Jobs, Send Continue Signals, Update Job Status, Maintain Job Table, Verify Execution, Test Background Recovery

```
root@DUDDUKURIs  ×  +  v
GNU nano 8.7.1      src/resume_job.c *
#include <stdio.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>

typedef struct {
    int job_id;
    pid_t pid;
    char state[20];
} Job;

int main() {
    Job job;
    pid_t pid;
    int status;

    pid = fork();

    if (pid < 0) {
        perror("fork");
        return 1;
    }

    if (pid == 0) {
        printf("Child: Job started. PID = %d\n", getpid());

        raise(SIGSTOP);

        printf("Child: Job resumed.\n");

        sleep(3);

        printf("Child: Job completed.\n");

        return 0;
    }
}
```

Output:

```
root@DUDDUKURIs:~/OS_Skill/Skill-Week-9$ gcc src/resume_job.c -o bin/resume_job
root@DUDDUKURIs:~/OS_Skill/Skill-Week-9$ ./bin/resume_job

Job table:
[1] PID=1316 State=Running
Child: Job started. PID = 1316

Job stopped.
[1] PID=1316 State=Stopped

Sending SIGCONT...
[1] PID=1316 State=Running
Child: Job resumed.
Child: Job completed.

Final job status:
[1] PID=1316 State=Completed
```